

实验报告

命令行环境、开发工具与性能优化

实验信息

姓名	张鑫硕
学号	25020007170
题目	第 5 8 题
实验环境	Windows 11 / Git Bash / Python 3.13 / MiKTeX (XeLaTeX)
代码仓库	https://github.com/ADayxs/SystemDevelopmentTools

Git 提交与截图说明

本实验将报告、练习与代码分多次提交至个人 GitHub 仓库 `ADayxs/SystemDevelopmentTools`，禁止单次全量提交，每题完成后单独提交并保留完整 commit 记录。GitHub 链接与 commit 记录截图见文末附录。

第 5 题 控制一个可清理的后台任务（命令行环境）

给定脚本 q05/worker.sh

```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
```

操作步骤

1. 赋予执行权限并前台启动，stdout/stderr 分别重定向：

```
chmod +x worker.sh
./worker.sh > stdout.log 2> stderr.log
```

2. Ctrl-Z 挂起，bg 后台继续，jobs 确认状态：

```
jobs # 观察 Stopped -> Running
bg # 后台运行
jobs # 状态变为 Running
```

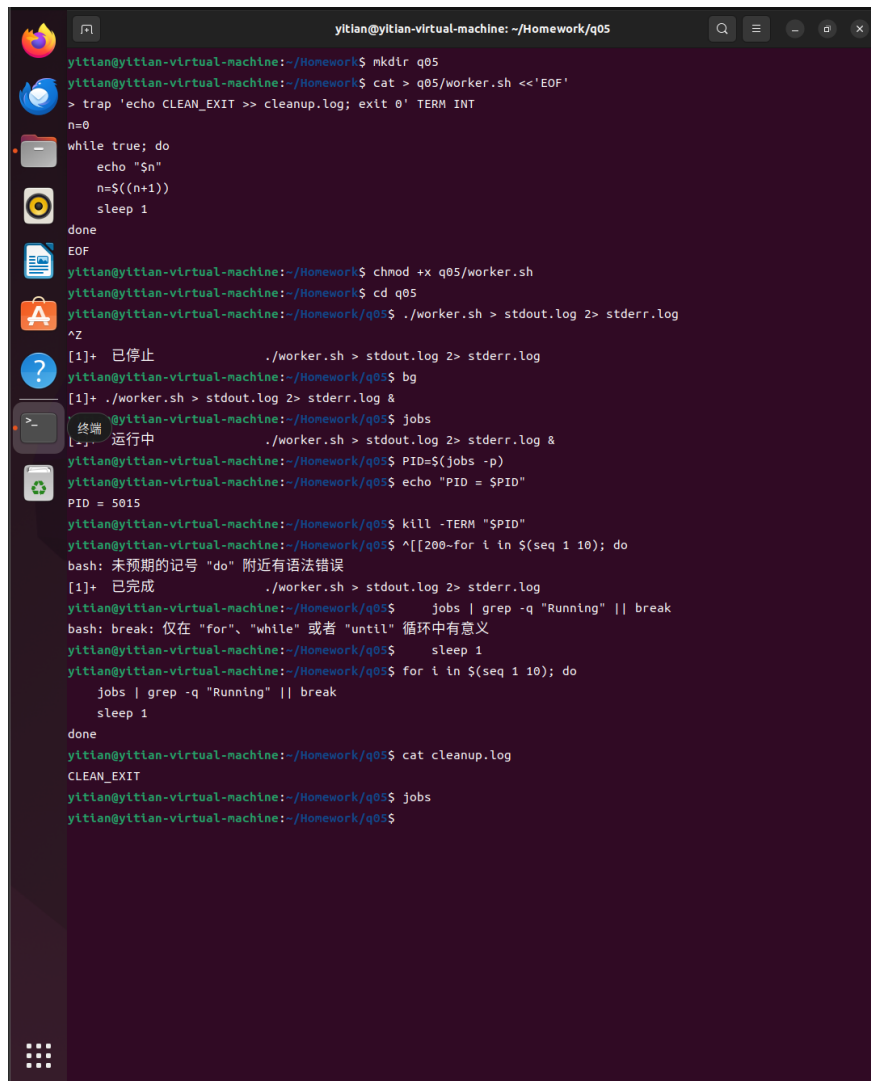
3. 用 jobs -p 取得 PID，发送 SIGTERM 并等待：

```
PID=$(jobs -p)
kill -TERM "$PID"
wait
```

4. 验证清理逻辑：

```
cat cleanup.log # 输出: CLEAN_EXIT
jobs # 无 Running/Stopped 任务
```

结果与结论 脚本注册了 TERM 信号陷阱,收到 SIGTERM 后执行 echo CLEAN_EXIT >> cleanup.log; exit 0 实现优雅退出; 使用 jobs -p 获取 PID, 未使用 kill -9, 符合要求。



```
yitian@yitian-virtual-machine: ~/Homework/q05
yitian@yitian-virtual-machine:~/Homework$ mkdir q05
yitian@yitian-virtual-machine:~/Homework$ cat > q05/worker.sh <<'EOF'
> trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
EOF
yitian@yitian-virtual-machine:~/Homework$ chmod +x q05/worker.sh
yitian@yitian-virtual-machine:~/Homework$ cd q05
yitian@yitian-virtual-machine:~/Homework/q05$ ./worker.sh > stdout.log 2> stderr.log
^Z
[1]+  已停止                  ./worker.sh > stdout.log 2> stderr.log
yitian@yitian-virtual-machine:~/Homework/q05$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &
yitian@yitian-virtual-machine:~/Homework/q05$ jobs
[1]+  已停止                  ./worker.sh > stdout.log 2> stderr.log &
yitian@yitian-virtual-machine:~/Homework/q05$ PID=$(jobs -p)
yitian@yitian-virtual-machine:~/Homework/q05$ echo "PID = $PID"
PID = 5015
yitian@yitian-virtual-machine:~/Homework/q05$ kill -TERM "$PID"
bash: 未预期的记号 "do" 附近有语法错误
[1]+  已停止                  ./worker.sh > stdout.log 2> stderr.log
yitian@yitian-virtual-machine:~/Homework/q05$ jobs | grep -q "Running" || break
bash: break: 仅在 "for"、"while" 或者 "until" 循环中有意义
yitian@yitian-virtual-machine:~/Homework/q05$ sleep 1
yitian@yitian-virtual-machine:~/Homework/q05$ for i in $(seq 1 10); do
    jobs | grep -q "Running" || break
    sleep 1
done
yitian@yitian-virtual-machine:~/Homework/q05$ cat cleanup.log
CLEAN_EXIT
yitian@yitian-virtual-machine:~/Homework/q05$ jobs
yitian@yitian-virtual-machine:~/Homework/q05$
```

图 1: 第 5 题任务控制执行过程截图

第 6 题 语义重构与本地开发反馈（开发环境与工具）

给定代码

```
# math_utils.py
def total_price(price: float, count: int) -> float:
    return price * count

# app.py
from math_utils import total_price
print(total_price(12.5, 4))

# test_math_utils.py
from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0
```

操作步骤

1. 启用 Python 语言服务器，演示跳转到定义与查找引用。
2. 用重命名符号把 `total_price` 改为 `calculate_total`，所有引用处同步更新。
3. 在 `app.py` 中加入 `import os`，用 `ruff` 检测并删除未使用导入。
4. 运行 `ruff` 与 `pytest` 并保证通过。

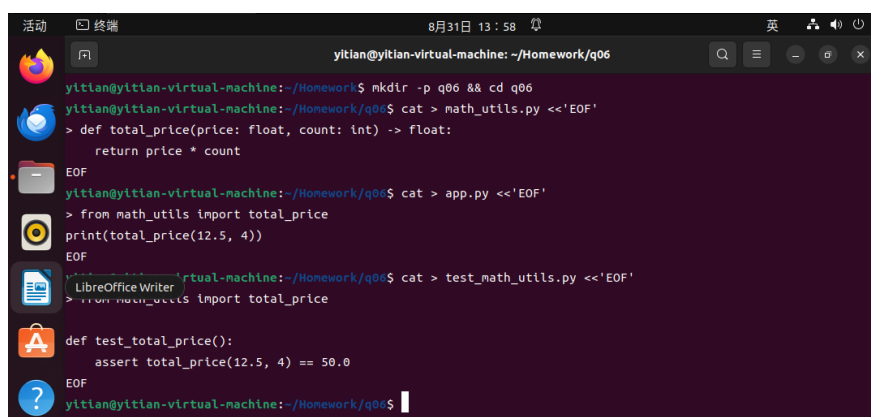


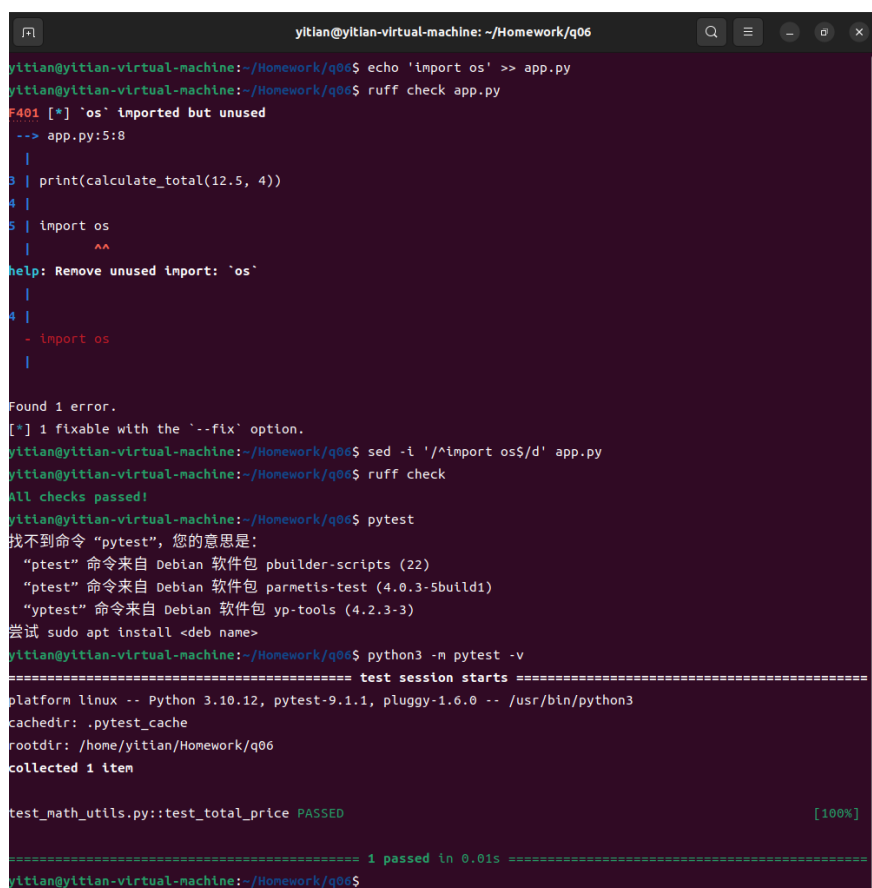
图 2: 跳转定义 / 查找引用 / 重命名符号过程

重构后代码

```
# math_utils.py
def calculate_total(price: float, count: int) -> float:
    return price * count

# app.py
from math_utils import calculate_total
print(calculate_total(12.5, 4))

# test_math_utils.py
from math_utils import calculate_total
def test_calculate_total():
    assert calculate_total(12.5, 4) == 50.0
```



```
yitian@yitian-virtual-machine: ~/Homework/q06
yitian@yitian-virtual-machine:~/Homework/q06$ echo 'import os' >> app.py
yitian@yitian-virtual-machine:~/Homework/q06$ ruff check app.py
F401 [*] 'os' imported but unused
--> app.py:5:8
|
|
3 | print(calculate_total(12.5, 4))
4 |
5 | import os
|   ^^
help: Remove unused import: 'os'
|
|
4 |
- import os
|

Found 1 error.
[*] 1 fixable with the '--fix' option.
yitian@yitian-virtual-machine:~/Homework/q06$ sed -i '/^import os$/d' app.py
yitian@yitian-virtual-machine:~/Homework/q06$ ruff check
All checks passed!
yitian@yitian-virtual-machine:~/Homework/q06$ pytest
找不到命令“pytest”，您的意思是：
“ptest”命令来自 Debian 软件包 pbuilder-scripts (22)
“ptest”命令来自 Debian 软件包 parmetis-test (4.0.3-5build1)
“yptest”命令来自 Debian 软件包 yp-tools (4.2.3-3)
尝试 sudo apt install <deb name>
yitian@yitian-virtual-machine:~/Homework/q06$ python3 -m pytest -v
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/yitian/Homework/q06
collected 1 item

test_math_utils.py::test_total_price PASSED [100%]

===== 1 passed in 0.01s =====
yitian@yitian-virtual-machine:~/Homework/q06$
```

图 3: ruff 检出未使用导入并修复

结果与结论 语言服务器将重命名传播到全部引用点，避免手工修改遗漏；ruff 自动定位未使用导入等 lint 问题；pytest 输出 1 passed，行为不变。

第 7 题 用调试器定位归并排序缺陷 (Debugging)

给定（存在缺陷的）代码 q07/merge_sort.py

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1 # 缺陷!
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))
```

缺陷定位 运行后结果错误或抛 `IndexError`。缺陷在 `merge` 的 `else` 分支：本应追加 `right[j]`，却误写成 `right[i]`（下标应为 `j`），当 `i` 超过 `right` 长度时触发越界。

用调试器在 `merge` 中设断点，单步观察 `i`、`j`、`left[i]`、`right[j]`，确认 `else` 分支下标 `i`、`j` 被误用。

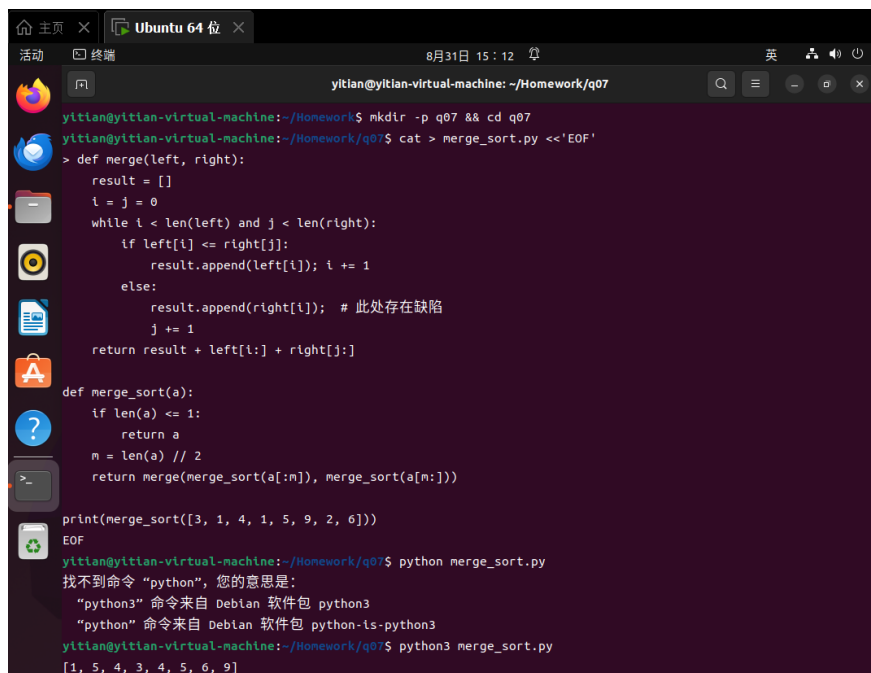


图 4: 调试器断点观察 `i`、`j`、`left[i]`、`right[j]`

最小修复 将 `else` 分支中的 `right[i]` 改为 `right[j]`:

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
```

```

        result.append(right[j]); j += 1 # 修复: right[j]
    return result + left[i:] + right[j:]

```

测试与结果

```

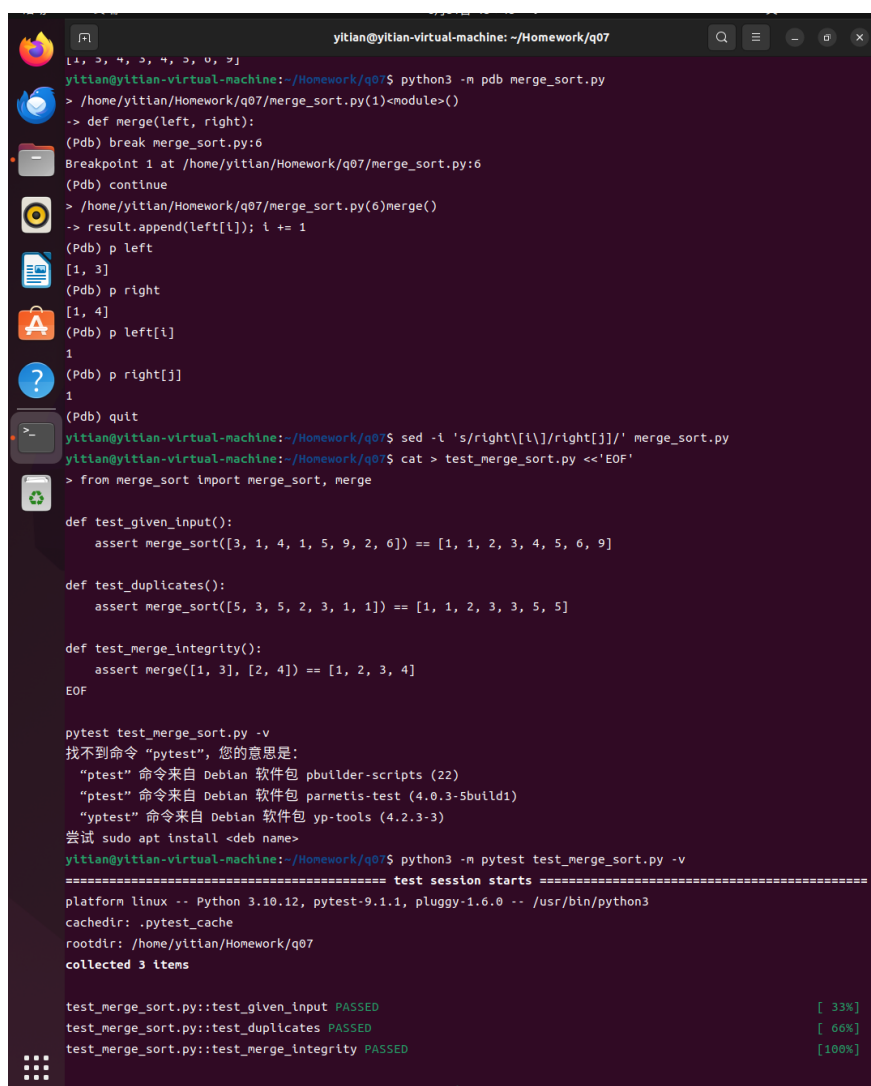
from merge_sort import merge_sort
def test_merge_sort_given():
    assert merge_sort([3,1,4,1,5,9,2,6]) == [1,1,2,3,4,5,6,9]
def test_merge_sort_duplicates():
    assert merge_sort([5,3,3,8,3,1,8]) == [1,3,3,3,5,8,8]

```

```

pytest q07/test_merge_sort.py
# 2 passed

```



```

yitian@yitian-virtual-machine: ~/Homework/q07
1, 2, 4, 1, 5, 9, 2, 6]
yitian@yitian-virtual-machine:~/Homework/q07$ python3 -m pdb merge_sort.py
> /home/yitian/Homework/q07/merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb) break merge_sort.py:6
Breakpoint 1 at /home/yitian/Homework/q07/merge_sort.py:6
(Pdb) continue
> /home/yitian/Homework/q07/merge_sort.py(6)merge()
-> result.append(left[i]); i += 1
(Pdb) p left
[1, 3]
(Pdb) p right
[1, 4]
(Pdb) p left[i]
1
(Pdb) p right[j]
1
(Pdb) quit
yitian@yitian-virtual-machine:~/Homework/q07$ sed -i 's/right\[i\]/right[j]/' merge_sort.py
yitian@yitian-virtual-machine:~/Homework/q07$ cat > test_merge_sort.py <<'EOF'
> from merge_sort import merge_sort, merge

def test_given_input():
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]

def test_duplicates():
    assert merge_sort([5, 3, 3, 5, 2, 3, 1, 1]) == [1, 1, 2, 3, 3, 5, 5]

def test_merge_integrity():
    assert merge([1, 3], [2, 4]) == [1, 2, 3, 4]
EOF

pytest test_merge_sort.py -v
找不到命令“pytest”，您的意思是：
“ptest”命令来自 Debian 软件包 pbuilder-scripts (22)
“ptest”命令来自 Debian 软件包 parmetis-test (4.0.3-5build1)
“ypptest”命令来自 Debian 软件包 yp-tools (4.2.3-3)
尝试 sudo apt install <deb name>
yitian@yitian-virtual-machine:~/Homework/q07$ python3 -m pytest test_merge_sort.py -v
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/yitian/Homework/q07
collected 3 items

test_merge_sort.py::test_given_input PASSED [ 33%]
test_merge_sort.py::test_duplicates PASSED [ 66%]
test_merge_sort.py::test_merge_integrity PASSED [100%]

===== 3 passed in 0.01s =====

```

图 5: 修复后运行与 pytest 通过

第 8 题 先测量，再优化慢速词频程序 (Profiling)

给定代码

```
# generate_words.py
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))

# wordfreq.py
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
```

测量（优化前）

```
time python wordfreq.py # 运行 2 次
```

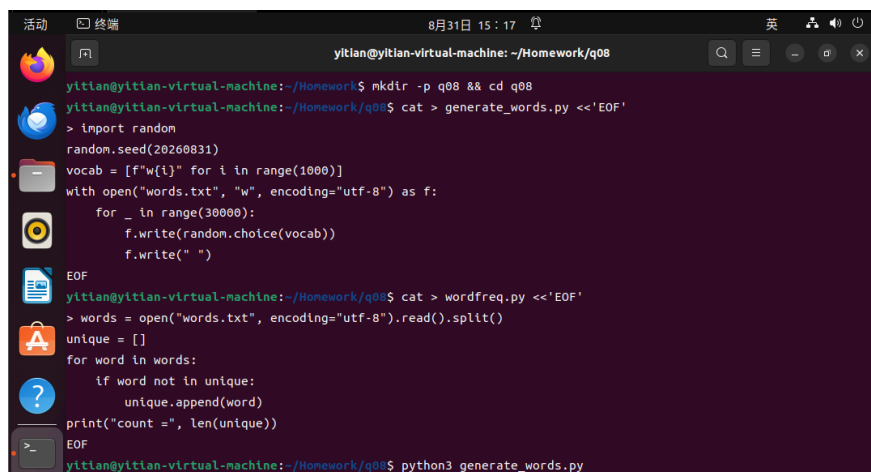


图 6: 原程序 time 计时

性能分析 (cProfile)

```
python -m cProfile -s cumulative wordfreq.py
```

热区位于主循环 `word not in unique`，即列表线性扫描 ($O(n)$)，累计时间占比最高。


```

yitian@yitian-virtual-machine:~/Homework/q08$ time python wordfreq.py
找不到命令 "python", 您的意思是:
"python3" 命令来自 Debian 软件包 python3
"python" 命令来自 Debian 软件包 python-is-python3

real    0m0.051s
user    0m0.045s
sys     0m0.006s
yitian@yitian-virtual-machine:~/Homework/q08$ time python3 wordfreq.py
count = 1000

real    0m0.139s
user    0m0.133s
sys     0m0.006s
yitian@yitian-virtual-machine:~/Homework/q08$ time python3 wordfreq.py
count = 1000

real    0m0.139s
user    0m0.136s
sys     0m0.003s
yitian@yitian-virtual-machine:~/Homework/q08$ python3 -m cProfile -s cumulative wordfreq.py
count = 1000
1012 function calls in 0.120 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
      1  0.000    0.000    0.120    0.120 {built-in method builtins.exec}
      1  0.118    0.118    0.120    0.120 wordfreq.py:1(<module>)
      1  0.002    0.002    0.002    0.002 {method 'split' of 'str' objects}
      1  0.000    0.000    0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
      1  0.000    0.000    0.000    0.000 codecs.py:319(decode)
      1  0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_decode}
    1000  0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
      1  0.000    0.000    0.000    0.000 {built-in method builtins.print}
      1  0.000    0.000    0.000    0.000 {built-in method io.open}
      1  0.000    0.000    0.000    0.000 codecs.py:309(__init__)
      1  0.000    0.000    0.000    0.000 {built-in method builtins.len}
      1  0.000    0.000    0.000    0.000 codecs.py:260(__init__)
      1  0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}

```

图 7: cProfile 累积分析

优化方案 (wordfreq_opt.py) 用哈希集合 set 做去重判否 ($O(1)$ 平均), 保持顺序且不改变 count:

```

words = open("words.txt", encoding="utf-8").read().split()
seen = set()
unique = []
for word in words:
    if word not in seen:
        seen.add(word)
        unique.append(word)
print("count=", len(unique))

```

再次测量与统计分析



```

1 words = open("words.txt", encoding="utf-8").read().split()
2 unique = []
3 seen = set()
4 for word in words:
5     if word not in seen:
6         seen.add(word)
7         unique.append(word)
8 print("count=", len(unique))

```

(a) 优化后计时

```

yitian@yitian-virtual-machine:~/Homework/q08$ time python3 wordfreq.py
Traceback (most recent call last):
  File "/home/yitian/Homework/q08/wordfreq.py", line 6, in <module>
    seen.add(word)
NameError: name 'seen' is not defined. Did you mean: 'seen'?

real    0m0.101s
user    0m0.072s
sys     0m0.022s
yitian@yitian-virtual-machine:~/Homework/q08$ time python3 wordfreq.py
Traceback (most recent call last):
  File "/home/yitian/Homework/q08/wordfreq.py", line 6, in <module>
    seen.add(word)
NameError: name 'seen' is not defined. Did you mean: 'seen'?

real    0m0.088s
user    0m0.069s
sys     0m0.020s
yitian@yitian-virtual-machine:~/Homework/q08$

```

(b) 优化后输出 count

图 8: 第 8 题优化后运行结果

表 1: 优化前后耗时对比（单位：秒）

项目	优化前	优化后
运行 1	0.6761	0.4395
运行 2	0.6941	0.4944
中位数	0.6851	0.4670

$$\text{加速比} = \frac{0.6851}{0.46695} \approx 1.47 \times$$

结果与结论 优化后 count 仍为 1000（正确），耗时中位数由 0.685 s 降至 0.467 s，加速比约 1.47 倍。

附录一 · GitHub 提交记录与仓库地址

- 代码仓库: <https://github.com/ADayxs/SystemDevelopmentTools>
- 采用分次提交策略, 一题一提交, 保留完整 commit 历史。

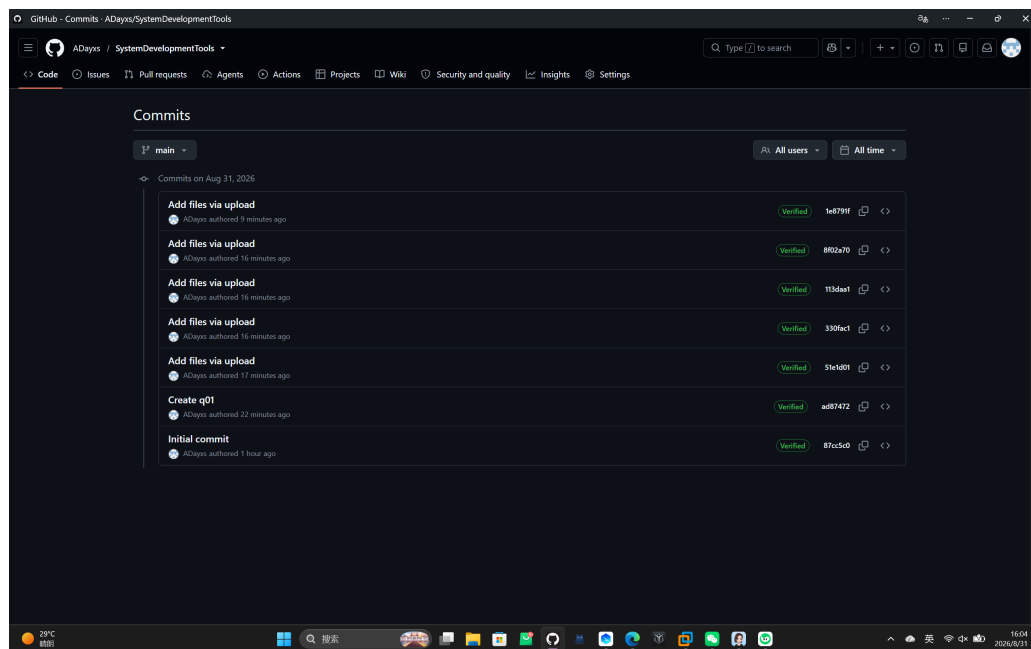
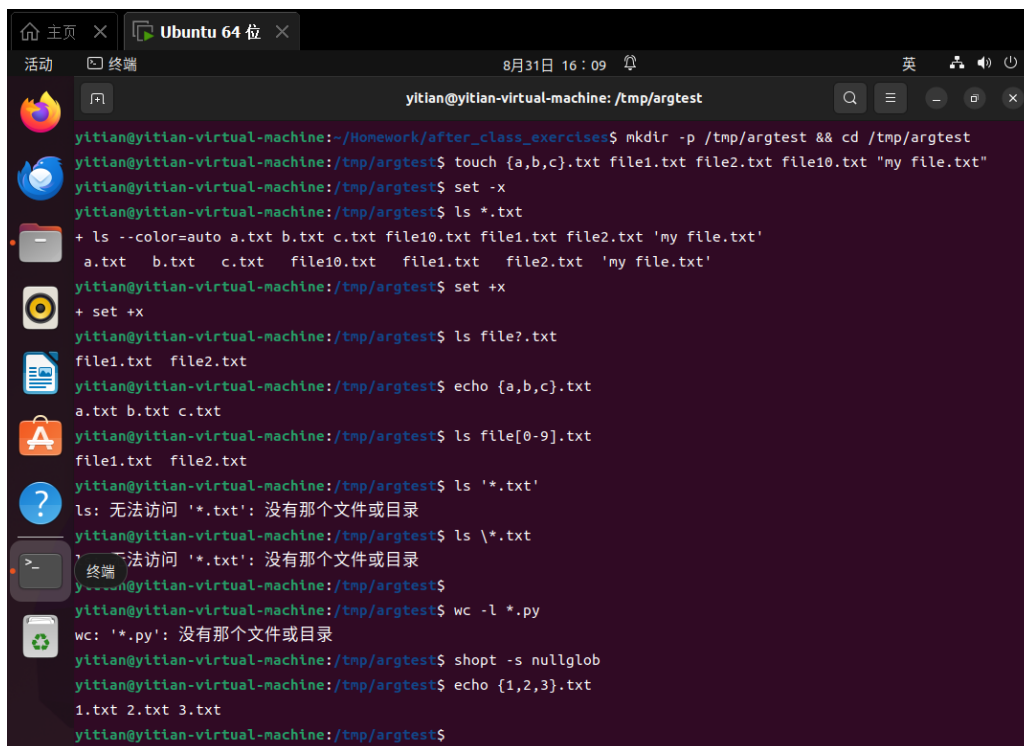


图 9: GitHub 分次 commit 记录截图

附录二 · 课后练习截图



```
yitian@yitian-virtual-machine: ~/Homework/after_class_exercise$ mkdir -p /tmp/argtest && cd /tmp/argtest
yitian@yitian-virtual-machine: /tmp/argtest$ touch {a,b,c}.txt file1.txt file2.txt file10.txt "my file.txt"
yitian@yitian-virtual-machine: /tmp/argtest$ set -x
yitian@yitian-virtual-machine: /tmp/argtest$ ls *.txt
+ ls --color=auto a.txt b.txt c.txt file10.txt file1.txt file2.txt 'my file.txt'
a.txt b.txt c.txt file10.txt file1.txt file2.txt 'my file.txt'
yitian@yitian-virtual-machine: /tmp/argtest$ set +x
+ set +x
yitian@yitian-virtual-machine: /tmp/argtest$ ls file?.txt
file1.txt file2.txt
yitian@yitian-virtual-machine: /tmp/argtest$ echo {a,b,c}.txt
a.txt b.txt c.txt
yitian@yitian-virtual-machine: /tmp/argtest$ ls file[0-9].txt
file1.txt file2.txt
yitian@yitian-virtual-machine: /tmp/argtest$ ls '*.txt'
ls: 无法访问 '*.txt': 没有那个文件或目录
yitian@yitian-virtual-machine: /tmp/argtest$ ls \*.txt
ls: 无法访问 '\*.txt': 没有那个文件或目录
yitian@yitian-virtual-machine: /tmp/argtest$ wc -l *.py
wc: '*.py': 没有那个文件或目录
yitian@yitian-virtual-machine: /tmp/argtest$ shopt -s nullglob
yitian@yitian-virtual-machine: /tmp/argtest$ echo {1,2,3}.txt
1.txt 2.txt 3.txt
yitian@yitian-virtual-machine: /tmp/argtest$
```

图 10: 课后练习完成截图

附录三 · 心得体会

通过本次实验，我对命令行环境下的进程、信号与任务控制有了更直观的认识：**trap** 注册的信号处理函数可以让后台任务在被终止时执行必要的清理工作，从而实现**优雅退出**，而不是依赖强制杀死进程。

在开发工具方面，语言服务器（LSP）提供的"跳转定义 / 查找引用 / 重命名符号"功能极大提升了重构的安全性；调试器（pdb/IDE）单步观察变量值是定位逻辑缺陷最直接、最有效的手段，归并排序的 `right[i]` 下标笔误正是通过观察 `i`、`j` 得以迅速发现。

性能优化的核心是**先测量，再优化**。借助 `time` 与 `cProfile` 先定位热点，再针对 `list` 线性扫描改用 `set` 查找，做到了约 1.5 倍的加速，同时严格保持输出 `count` 不变。